fieldnote

AI Crop Disease Prediction & Management (P05)
Full improvement plan: bugs, fixes, model quality, evaluation, demo

1. Project summary

What the project must do (from your P05 brief): an image + environmental-risk model for early crop disease prediction. Dataset: PlantVillage plus separately sourced public weather/environmental data, with explicit spatial/temporal alignment. Synthetic environmental variables may be used for controlled simulation only where real linkage is unavailable.

What you have built: a FastAPI backend with a MobileNetV2 image classifier, a rule-based environmental risk score, a fusion engine, a static knowledge base, Open-Meteo weather lookup, and a plain HTML/JavaScript front end (fieldnote UI).

Important note: I have not seen your code, only your written analysis and the UI screenshot. The code in this document shows patterns to adapt to your file names and structure. If you send the real files, I can turn each pattern into an exact patch.

2. What is already good

- The UI states honestly that results are "guidance, not a confirmed diagnosis" and that the score is "not a calibrated probability". Keep this.
- Clear separation of vision, environment and fusion in the API response (`vision_details`, `environmental_details`, `multimodal_fusion`).
- Upload size limit (10 MiB) and rejection of incomplete location/date metadata.
- Weather comes from a real source (Open-Meteo) with a labeled leaf-wetness estimate, which supports the P05 requirement for spatial/temporal alignment.
- The dataset metadata file already documents the 38-class split versus the 53-label checkpoint.

3. Findings at a glance

#	Issue	Severity	Effect on users / project	Fix
1	`runDiagnosis()` uses `vision, environment, fusion` that only exist inside `renderResults()`	High	Results show, then the UI reports failure; ready state and chat context never set	Return values from render; separate request errors from render errors
2	Checkpoint has 53 labels, but only 10 match knowledge-base profiles	High	43 labels get generic advice; weak answers	Alias map to 38 canonical classes; one KB entry per class; test
3	No uncertainty handling	High	Wrong confident answers on real field photos	Confidence threshold and top-3 output
4	Image dimensions not bounded before decode	Medium	Decompression-bomb / memory risk	Check pixels before convert/resize
5	CORS allows all origins with credentials	Medium	Invalid and unsafe configuration	`allow_credentials=False` or explicit origins

#	Issue	Severity	Effect on users / project	Fix
6	Model loads at import time	Medium	Tests cannot run without loading weights	Load in FastAPI lifespan; stub in tests
7	Simulated vs real weather not always obvious	Medium	Risks breaking P05 spec compliance	Label weather source in API and UI
8	Fusion weights are not explained	Low	Hard to justify in demo	Return per-part contributions
9	No held-out evaluation reported	Medium	No evidence the model works	Per-class accuracy, confusion matrix, leak-free split

4. Detailed fixes (in recommended order)

Fix 1 - Frontend scope bug in app.js

Cause: `vision`, `environment` and `fusion` are declared inside `renderResults()`, but `runDiagnosis()` reads them afterwards. That throws a `ReferenceError`, and the catch block shows a failure message even though the prediction succeeded.

Fix: make `renderResults()` return the values, and wrap only the network call in the try/catch so that a rendering bug is not reported as a failed prediction.

```
function renderResults(data) {
  const vision      = data.vision_details;
  const environment = data.environmental_details;
  const fusion      = data.multimodal_fusion;
  // ... write values into the DOM ...
  return { vision, environment, fusion };
}

async function runDiagnosis() {
  let data;
  try {
    data = await postFused(formData);      // network/API errors only
  } catch (err) {
    return showError("Prediction request failed.", err);
  }
  try {
    const ctx = renderResults(data);
    state.chatContext = ctx;               // used by follow-up chat
    setReady(true);
  } catch (err) {
    console.error(err);
    showError("Result could not be displayed.", err);
  }
}
```

How to check: upload one photo; the result card appears, the status chip changes to ready, and a follow-up question uses the new context.

Fix 2 - Label mismatch (53 labels vs 38 classes)

This is the biggest quality problem. Your checkpoint has naming variants of the same class, so most predictions miss their knowledge-base profile and fall back to generic text.

- Print all 53 labels from the class map and group the ones that mean the same thing.
- Create one alias dictionary that maps every label to one of the 38 canonical PlantVillage class names.
- Apply the alias function right after the model output, before the knowledge-base lookup.
- Write one knowledge-base entry per canonical class (38 entries): description, typical symptoms, favorable conditions, what to inspect next, general management steps.
- Add a test that fails if any class has no profile.

```
# class_aliases.py
CANONICAL = [  # the 38 PlantVillage classes
    "Apple___Apple_scab", "Apple___Black_rot",
    "Tomato___Early_blight", "Tomato___Late_blight",
    # ... all 38 ...
]
ALIASES = {
    "Tomato_Early_blight":  "Tomato___Early_blight",
    "tomato early blight":  "Tomato___Early_blight",
    # ... one line per extra label in the 53-label map ...
}

def canonical(label: str) -> str:
```

```
    c = ALIASES.get(label.strip(), label.strip())
    if c not in CANONICAL:
        raise ValueError(f"Unmapped label: {label}")
    return c

# tests
def test_all_model_labels_map():
    for lbl in MODEL_CLASS_MAP.values():
        canonical(lbl)

def test_every_class_has_profile():
    missing = [c for c in CANONICAL if c not in KNOWLEDGE_BASE]
    assert not missing, missing
```

Tip: if the 53-label checkpoint was trained with extra classes, either retrain on the 38-class curated split (cleaner and matches your dataset metadata), or keep the checkpoint and document the mapping. Do not mix both without saying which one you use.

Fix 3 - Image safety (dimension check)

The 10 MiB limit bounds the file size but not the decoded size. A small compressed file can decode to a huge image. Check the dimensions before converting.

```
from PIL import Image, UnidentifiedImageError
import io
from fastapi import HTTPException

MAX_PIXELS = 25_000_000
Image.MAX_IMAGE_PIXELS = MAX_PIXELS

def load_image(data: bytes) -> Image.Image:
    try:
        img = Image.open(io.BytesIO(data))      # lazy: header only
        if img.width * img.height > MAX_PIXELS:
            raise HTTPException(413, "Image dimensions too large")
        img.load()
        return img.convert("RGB")
    except (Image.DecompressionBombError, UnidentifiedImageError, OSError):
        raise HTTPException(400, "Invalid or unsupported image")
```

Also verify the content type is JPG, PNG or WEBP by decoding (as above), not only by file extension.

Fix 4 - CORS

Browsers reject `Access-Control-Allow-Origin: *` together with credentials. Because the app serves the front end itself and uses no cookies, the simplest correct setting is to turn credentials off.

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://127.0.0.1:8000", "http://localhost:8000"],
    allow_credentials=False,
    allow_methods=["GET", "POST"],
    allow_headers=["*"],
)
```

Fix 5 - Load the model in a lifespan handler

Loading MobileNetV2 when the module is imported makes tests slow and forces every import to need the weights. Load once at startup and store it on the app.

```
from contextlib import asynccontextmanager
from fastapi import FastAPI, Request

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.classifier = load_classifier()   # reads checkpoint once
    yield
```

```
app = FastAPI(lifespan=lifespan)

# in routes
def get_classifier(request: Request):
    return request.app.state.classifier

# in tests: override with a stub
class StubClassifier:
    def predict(self, img):
        return [("Tomato___Early_blight", 0.82),
                ("Tomato___Late_blight", 0.10),
                ("Tomato___healthy", 0.04)]
```

5. What makes it work very well

You said you want few features but high quality. These items improve reliability, not feature count.

A. Uncertainty handling (most important)

PlantVillage images are clean lab photos on plain backgrounds. Real field photos differ, so the model will sometimes be confidently wrong. Return an explicit low-confidence state instead of a bad guess.

```
CONF_MIN = 0.50      # tune on validation data
MARGIN_MIN = 0.15    # gap between top-1 and top-2

def decide(probs):   # probs: list of (label, prob) sorted high to low
    top1, top2 = probs[0], probs[1]
    low = top1[1] < CONF_MIN or (top1[1] - top2[1]) < MARGIN_MIN
    return {
        "status": "low_confidence" if low else "ok",
        "top3": [{"label": canonical(l), "score": round(p, 3)}
                 for l, p in probs[:3]],
        "message": ("Not confident. Retake a clear, close photo of one "
                    "leaf in daylight.") if low else None,
    }
```

In the UI, show the top-3 list and the retake message when status is `low_confidence`, and do not show a single bold disease name.

B. Fusion transparency

Return how much each part contributed so you can explain the result in your demo. State clearly that the weights are a heuristic and not calibrated.

```
W_VISION, W_ENV = 0.65, 0.35        # heuristic, documented, not calibrated

def fuse(vision_score, env_risk, is_healthy):
    v = 0.0 if is_healthy else vision_score
    v_part, e_part = W_VISION * v, W_ENV * env_risk
    return {
        "risk_index": round(100 * (v_part + e_part)),
        "vision_contribution": round(100 * v_part),
        "environment_contribution": round(100 * e_part),
        "note": "Heuristic index, not a calibrated probability.",
    }
```

Show two small bars in the UI: "Photo evidence" and "Field conditions", so a viewer sees why the index is high or low.

C. P05 compliance: label weather source every time

- Keep requiring location and date so weather is aligned in space and time.
- Add `weather_source` to the API response with value `open-meteo` or `simulated`.
- When the value is `simulated`, show a visible "Simulated weather" tag in the UI and in any report or export.
- Keep the leaf-wetness value labeled as an estimate, not a measurement.
- Describe in your report exactly when simulation is used and why real linkage was unavailable (this is what the brief allows).

```
"environmental_details": {
    "weather_source": "open-meteo",       # or "simulated"
    "location": {"lat": 31.63, "lon": 74.87},
    "date": "2026-10-27",
    "leaf_wetness_estimate": {"value": 0.62, "kind": "estimated"},
    "risk_score": 0.48,
    "risk_method": "rule-based, not calibrated"
```

```
}
```

D. Keep the knowledge base honest

- Write general, cautious management steps (inspect, remove badly affected leaves, improve airflow, avoid overhead watering, consult a local agriculture officer).
- Do not give pesticide names or doses; these depend on crop, region and local rules. Tell the user to confirm with a local expert.
- Keep the sentence "guidance, not a confirmed diagnosis" on every result.

6. Honest evaluation

A project reviewer will ask: how do you know it works? Prepare these numbers.

- **Leak-free split:** split by source image before augmentation, so augmented copies of one photo never appear in both train and test. PlantVillage has many near-duplicate images; if possible group by leaf or source file.
- **Metrics:** overall accuracy, per-class precision/recall/F1, and a confusion matrix on the held-out test set.
- **Confidence check:** plot accuracy against confidence bins to choose `CONF_MIN`. If high-confidence predictions are not more accurate, say so.
- **Out-of-distribution test:** try 20-30 real field photos (phone camera, soil and hands in background). Report the drop in accuracy honestly; this is a strong point in a report because it shows you understand the limit.
- **Environment score check:** since it is rule-based, show a small table of example inputs (humidity, temperature, rain, leaf wetness) and the resulting score so the logic can be inspected.

```
from sklearn.metrics import classification_report, confusion_matrix
import numpy as np

y_true, y_pred, y_conf = [], [], []
for img, label in test_loader:
    probs = model_predict(img)
    y_true.append(label); y_pred.append(probs.argmax()); y_conf.append(probs.max())

print(classification_report(y_true, y_pred, target_names=CANONICAL))
cm = confusion_matrix(y_true, y_pred)
np.save("confusion_matrix.npy", cm)
```

7. Testing plan

Test	What it proves
Stub-classifier API test	Endpoint returns vision, environment and fusion fields without loading weights
Label mapping test	Every model label maps to a canonical class
Knowledge-base test	Every canonical class has a profile
Oversized image test	Image above 25 MP returns 413; a 10 MiB+ file returns the size error
Corrupt file test	Non-image bytes return 400, not 500
Missing location/date test	Request is rejected with a clear message
Low-confidence test	Stub with scores 0.34/0.31/0.20 returns status low_confidence
Weather failure test	If Open-Meteo is unreachable, the API returns a clear message or labeled simulated data, never a silent default

Test	What it proves
Frontend manual test	Upload success, upload failure, low-confidence, chat after result

8. Work order (suggested time plan)

Step	Task	Time
1	Fix `app.js` scope bug	30 min
2	Lifespan model loading and stub test setup	1 hour
3	Image dimension check and CORS fix	30 min
4	Label alias map and 38 knowledge-base entries	3-4 hours
5	Confidence threshold and top-3 UI	1-2 hours
6	Weather-source labeling and fusion contribution bars	1-2 hours
7	Evaluation script, confusion matrix, field-photo test	2-3 hours
8	Write report section and rehearse demo	2 hours

9. Demo checklist

- Show a clear leaf photo with a correct, confident result, with the fusion bars.
- Show a poor or unrelated photo and the low-confidence "retake photo" message.
- Show the same photo with two different weather dates to prove the environment score changes.
- Show a result with weather labeled Simulated and explain why (P05 rule).
- Ask a follow-up chat question to show the context works after the bug fix.
- Show the confusion matrix and the field-photo accuracy drop, and say what you would do next (more field data, fine-tuning).
- End with the limits: guidance only, heuristic fusion, not calibrated.

10. Known limitations to write in your report

- The image model was trained on lab-style PlantVillage photos and may not generalize to field conditions.
- The environmental score is rule-based, and the fusion weights are heuristic; neither is a calibrated probability.
- Leaf wetness is estimated from weather variables, not sensed on the leaf.
- The knowledge base is static text, not a database, and covers general guidance only.
- Results support deciding what to inspect next; they are not a confirmed diagnosis.

11. What I need to give exact patches

- `app.js`: `runDiagnosis` and `renderResults`
- `fusion_engine.py` and `environmental_risk.py`
- `vision_classifier.py`: the class-map and prediction section
- `routes.py`: the predict route and image loading

- The list of the 53 labels (print the class map)